CodeGate Security Scan Report

Target: examples/vulnerable_app

Scanned: 2026-10-05T08:13:30.136230+00:00

CodeGate version: 0.2.0

Ranker: R2 | Corpus: real-2040-sym | Verifier: not used

13 finding(s) across 14 scanned function(s), 0 file(s) skipped.

Executive Summary

14 function(s) scanned. 4 auto-approved under the certificate. 10 escalated for review, of which 0 were sent to the LLM verifier this run and 4 were resolved without one (auto-approved or already cached).

Severity	Count
ERROR	2
REVIEW	10
WARNING	1

Detection Attribution

Source	Count	What it means
Semgrep (static rules)	3	Deterministic pattern match, zero LLM cost
ML gate (escalated)	10	Model flagged as not-certifiably-safe
LLM verifier (confirmed)	0	Of the escalated units, confirmed vulnerable
Needs manual review	10	Escalated, not resolved by the verifier
Auto-approved	4	Cleared under the certified miss-rate <= alpha

Cost & Token Accounting

Metric	Value
Tokens saved (est.)	132
Tokens spent (est.)	0
LLM calls avoided	4
LLM calls made	0
Estimated cost	$0.0000
Price used	$0.59 / 1M tokens

All figures on this page are estimates. Token counts use a chars/4 heuristic (no tiktoken dependency), not an exact tokenizer count; "tokens saved" is measured against a baseline of sending every scanned function to the LLM verifier exactly once. See docs/ENGINEERING_LOG.md for the full methodology.

Findings Detail

examples/vulnerable_app/app.py:8 -- handle_login()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.33

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 5):

```
from system_tasks import ping_host


def handle_login(username):
```

```
    conn = seed_demo_data(sqlite3.connect(":memory:"))
    return get_user_by_name(conn, username)
```

examples/vulnerable_app/app.py:13 -- handle_diagnostics()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.25

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 10):

```
    return get_user_by_name(conn, username)


def handle_diagnostics(host):
    return ping_host(host)
```

examples/vulnerable_app/config.py:5 -- <module>()

CWE-798 (Hardcoded Credentials) -- severity WARNING, source semgrep, confidence 1.00

String literal assigned to a variable whose name looks like a credential (password/secret/token/api_key/access_key).

Excerpt (from line 2):

```
# Vulnerable: hardcoded credential literal (CWE-798) -- ships with every
# clone of this repo and can't be rotated without a code change.
DATABASE_PASSWORD = "Sup3rSecretPassw0rd!"

APP_NAME = "vulnerable_app_demo"  # Safe: not a credential.
```

Remediation:

A credential literal in source code ships with every clone of the repository and every deployed artifact -- it is readable by anyone with source access, persists in version-control history even after being removed, and cannot be rotated without a code change and redeploy. Loading it from an environment variable or a secret store keeps it out of source entirely and lets it be rotated independently of the code.

Before:

```
password = "Sup3rSecretPassw0rd!"
```

After:

```
password = os.environ["DB_PASSWORD"]
```

examples/vulnerable_app/database.py:4 -- get_user_by_name()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.34

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 1):

```
import sqlite3


def get_user_by_name(conn, username):
    """Vulnerable: SQL query built by string concatenation (CWE-89)."""
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    return conn.execute(query).fetchone()
```

examples/vulnerable_app/database.py:6 -- get_user_by_name()

CWE-89 (SQL Injection) -- severity ERROR, source semgrep, confidence 1.00

SQL query text built by concatenation/formatting with a variable, then executed. Use a parameterized query instead.

Excerpt (from line 3):

```
def get_user_by_name(conn, username):
    """Vulnerable: SQL query built by string concatenation (CWE-89)."""
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    return conn.execute(query).fetchone()
```

Remediation:

Building a SQL query by concatenating or formatting untrusted input directly into the query text lets an attacker change the query's structure -- for example closing a string literal early to inject their own WHERE clause or UNION SELECT. A parameterized query keeps the query's structure fixed and passes values through a separate channel the database driver escapes correctly, so no input can change what the query does.

Before:

```
query = "SELECT * FROM users WHERE username = '" + username + "'"
cur = conn.execute(query)
```

After:

```
cur = conn.execute(
    "SELECT * FROM users WHERE username = ?", (username,)
)
```

---

examples/vulnerable_app/database.py:10 -- get_user_by_id()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.21

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 7):

```
    return conn.execute(query).fetchone()


def get_user_by_id(conn, user_id):
    """Safe: parameterized query -- the fix for get_user_by_name above."""
    return conn.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()
```

---

examples/vulnerable_app/database.py:15 -- list_active_users()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.27

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 12):

```
    return conn.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()


def list_active_users(conn):
    """Safe: no external input reaches the query at all."""
    return conn.execute("SELECT * FROM users WHERE active = 1").fetchall()
```

---

examples/vulnerable_app/database.py:20 -- seed_demo_data()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.33

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 17):

```
    return conn.execute("SELECT * FROM users WHERE active = 1").fetchall()


def seed_demo_data(conn):
    conn.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, active INTEGER)")
    conn.execute("INSERT INTO users VALUES (1, 'alice', 1)")
    conn.commit()
```

examples/vulnerable_app/report_utils.py:6 -- slugify()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.26

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 3):

```
    return celsius * 9 / 5 + 32


def slugify(text):
    """Safe: string formatting only."""
    return text.lower().strip().replace(" ", "-")
```

examples/vulnerable_app/system_tasks.py:5 -- ping_host()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.34

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 2):

```
import subprocess


def ping_host(host):
    """Vulnerable: shell=True lets shell metacharacters in `host` run
    arbitrary commands (CWE-78) -- e.g. host="1; rm -rf ~"."""
    return subprocess.run(f"ping -c 1 {host}", shell=True, capture_output=True)
```

examples/vulnerable_app/system_tasks.py:8 -- ping_host()

CWE-78 (OS Command Injection) -- severity ERROR, source semgrep, confidence 1.00

Subprocess invoked with shell=True (or os.system/os.popen). If any argument is influenced by external input this is command injection.

Excerpt (from line 5):

```
def ping_host(host):
    """Vulnerable: shell=True lets shell metacharacters in `host` run
    arbitrary commands (CWE-78) -- e.g. host="1; rm -rf ~"."""
    return subprocess.run(f"ping -c 1 {host}", shell=True, capture_output=True)


def ping_host_safely(host):
```

Remediation:

shell=True (or os.system/os.popen) runs the command through a shell, which interprets metacharacters -- ;, |, &&, $(), backticks -- in any input that reaches the command string. Untrusted input can then chain or substitute

arbitrary commands. Passing the command as a list of arguments with shell=False invokes the program directly, with no shell present to interpret metacharacters at all.

Before:

```
subprocess.run(f"ping -c 1 {host}", shell=True)
```

After:

```
subprocess.run(["ping", "-c", "1", host], shell=False)
```

---

examples/vulnerable_app/system_tasks.py:11 -- ping_host_safely()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.20

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 8):

```
    return subprocess.run(f"ping -c 1 {host}", shell=True, capture_output=True)


def ping_host_safely(host):
    """Safe: the fix for ping_host above -- an argv list, no shell involved."""
    return subprocess.run(["ping", "-c", "1", host], shell=False, capture_output=True)
```

---

examples/vulnerable_app/system_tasks.py:16 -- list_directory()

UNSPECIFIED (Needs manual review) -- severity REVIEW, source model, confidence 0.39

Not certifiably safe under the current calibration -- needs LLM verification.

Excerpt (from line 13):

```
    return subprocess.run(["ping", "-c", "1", host], shell=False, capture_output=True)


def list_directory(path):
    """Safe: no subprocess or shell at all."""
    return os.listdir(path)
```

Coverage & Certificate

Field	Value
Ranker	R2
Corpus version	real-2040-sym
Provisional	no
t_hat (auto-approve threshold)	0.8023
Certified max miss-rate (alpha)	10.00%
Confidence (1 - delta)	95.00%
Calibration vulnerable samples	218
Certificate method	cluster-aware: 66 task groups (Hoeffding-Bentkus)
Created	2026-10-05T07:55:15.953676+00:00

What this guarantees: averaged over coding tasks, at most 10.00% of a task's vulnerable functions score at or above t_hat and are auto-approved, with 95.00% confidence -- a distribution-free certificate (Learn-Then-Test conformal risk control), not a heuristic claim. Each task group in the calibration data counts as one piece of evidence, however many models answered it, so correlated answers to the same task cannot inflate the certificate. It does not guarantee anything about a function scored below t_hat (escalated) beyond "not certified safe."

Methodology & Limitations

CodeGate is a certified-miss-rate security scanner for Python. It escalates to an LLM only what it cannot statistically prove is safe, using Learn-Then-Test conformal risk control over a ranker trained on labeled generated code.

Scope: Python only. Three CWEs are tracked -- CWE-89 (SQL injection), CWE-78 (OS command injection), CWE-798 (hardcoded credentials). Analysis is intraprocedural (one function at a time); cross-function data flow is not modeled. The threat model is benign-accident, not adversarial -- the scanner never executes the code it scans.

Static findings (semgrep) are deterministic pattern matches against a hand-written local ruleset. Model findings come from a ranker scored against a certified threshold; unresolved model findings needed LLM verification that either wasn't attempted (--no-llm or no API key) or didn't return a usable verdict.